PROJECT REPORT

Hospital Management System

C++17 Console Application for OOP and Data Structures

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology

Submitted by

[Student Name] | Roll No. [Roll Number]

Under the guidance of

[Faculty Guide Name]

[Department Name]
[College Name]
Academic Year [20XX-20XX]

Certificate

This is to certify that the project report titled “Hospital Management System” is submitted by [Student Name], Roll No. [Roll Number], in partial fulfillment of the requirements for the Bachelor of Technology degree in [Department Name] during the academic year [20XX-20XX]. The work has been completed under my guidance for academic evaluation.

Faculty Guide Signature: _____
Name: [Faculty Guide Name]
Designation: [Designation]
Date: [Date]

Acknowledgement

I express sincere gratitude to [Faculty Guide Name], [Department Name], for guidance and support during this project. I also thank [College Name], my classmates, and my family for their encouragement. This report documents an adapted and verified academic software project; the original repository attribution is retained in the project documentation.

Declaration

I declare that this report describes work submitted for academic evaluation. Names, roll numbers, approvals, and signatures in this template must be completed by the submitting student and faculty before submission.

Contents

Abstract

1. Introduction

2. Problem Statement Objectives and Scope

3. Requirements and Technology

4. System Architecture and Modules

5. Object Oriented Design

6. Data Structures and Algorithms

7. File Handling and Persistence

8. Testing and Results

9. Limitations and Future Scope

10. Conclusion

References

Abstract

The Hospital Management System is a C++17 console application developed to demonstrate object-oriented programming and core data structures in a hospital-workflow context. It provides administrator login, staff directory management, priority appointments, admissions and nurse notes, emergency case handling, hospital route finding, prefix search, feedback recording, shift rotation, and session viewing history. The project uses local CSV files for selected records and CMake for portable builds. It is designed for classroom demonstration and learning, not for use with real patient data or clinical decisions.

1. Introduction

Hospitals manage many records and workflow queues. This academic project models a selected subset of those tasks in a single-user console program. The implementation focuses on choosing data structures that match the required operations, such as balanced lookup for staff, priority processing for appointments, and shortest-path navigation between locations.

2. Problem Statement Objectives and Scope

Problem Statement

Manual or unstructured handling of staff, appointments, admissions, and emergency records can make it difficult to demonstrate efficient search, prioritization, and record flow. The project provides a compact simulation that connects these workflows to appropriate C++ data structures.

Objectives

Objective	Implemented evidence
Manage staff records	AVL-backed staff directory with add, list, delete, and file loading.
Prioritize appointments	Binary max-heap keeps high-priority appointments first.
Support patient-care flow	Check-up can move a patient to admission or session history.
Handle urgent cases	Stack-based emergency registration, triage display, discharge, and persistence.
Guide users through a hospital	Graph and Dijkstra algorithm calculate predefined routes.
Demonstrate reliable input	Required text and integer validation reduce broken console flows.

Scope

The scope is an academic console application with fictional data. It includes selected workflows and local CSV persistence. It excludes real clinical decision making, regulatory compliance, multi-user access, encrypted storage, and production billing rules.

3. Requirements and Technology

Category	Requirement
Hardware	Any modern macOS, Linux, or Windows computer capable of compiling C++17.
Operating system	Tested with macOS and Apple Clang in the project audit.
Language	C++17.
Build system	CMake 3.10 or later.
Compiler	A compiler with C++17 support, such as Apple Clang, Clang, GCC, or MSVC.
Storage	Local CSV files under the data directory.

4. System Architecture and Modules

HospitalApp acts as the controller. It handles login and central-dashboard routing, then delegates each workflow to focused modules. This separation keeps menu flow, storage, algorithms, billing output, and logging responsibilities understandable for a classroom demonstration.

Module	Responsibility	Key implementation
HospitalApp	Login and dashboard routing	Central controller and menu loops
HospitalDirectory	Staff records	AVL tree indexed by staff ID
PriorityQueue	Appointments and admissions	Max-heap, hash map, stack, queue
EmergencyManagement	Emergency cases	Current and discharged stacks with CSV state
HospitalGraph	Hospital routes	Adjacency list and Dijkstra shortest path
Trie	Prefix search	Predefined departments and doctor names
PatientHistory	Check-up history	Doubly linked list for back and next
ShiftRotator	On-call rotation	Circular linked list
Logger and BillingService	Support services	Mutex-protected log and invoice formatting

Main User Flow

The user opens the application, passes the login screen, and reaches the Central Dashboard. From there the user can navigate to staff administration, patient care, nursing, emergency response, navigation, search, history, or password settings. On normal exit, modules write their selected records to local CSV files.

5. Object Oriented Design

Concept	Use in the project
Encapsulation	Classes keep their internal structures private and expose focused operations.
Composition	HospitalApp owns and coordinates independent service classes.
Abstraction	Menus call operations such as bookAppointment and findShortestPath without exposing implementation details.
Resource management	Shared pointers manage dynamic records and destructors save selected state at normal shutdown.
Single responsibility	Directory, emergency, graph, trie, UI, logging, and billing responsibilities are separated.

6. Data Structures and Algorithms

Structure or algorithm	Application use	Typical complexity
AVL tree	Staff records by identifier	Search, insert, delete: O(log n)
Binary max-heap	Priority appointments	Insert/remove: O(log n), next: O(1)
Hash map	Admitted patient lookup	Average lookup: O(1)
Stacks	Emergency and old-patient records	Push/pop: O(1)
Queue	Feedback order	Enqueue/dequeue: O(1)
Trie	Prefix search	Prefix traversal: O(L)
Graph plus Dijkstra	Shortest route between locations	O((V + E) log V)
Doubly linked list	Current-session viewing history	Move back/next: O(1)
Circular linked list	Doctor shift rotation	Advance: O(1)

7. File Handling and Persistence

Selected state is saved under the data directory during a normal application shutdown and loaded at startup when files exist. Staff, appointments, admissions, complaints, active emergency records, discharged emergency records, and an optional local password file have dedicated CSV or text files. Input text is sanitized before CSV output by replacing commas with spaces. The design is suitable for a small classroom demo but does not provide transactions, encryption, or concurrent multi-user protection.

File	Stored content
hospital_records.csv	Staff identifiers, names, specialities, and surgeon status
appointments.csv	Active appointments and their check-up fields
admitted.csv	Admitted patient records and notes
complaints.csv	Feedback in first-in, first-out order
emergency_current.csv and emergency_old.csv	Active and discharged emergency records
password.txt	Demo password in plain text if changed locally

8. Testing and Results

Testing used fictional records. The project contains no registered CTest suite, so verification combined a CMake build with scripted console smoke tests. The final demo-readiness audit confirmed the application builds and reaches the Central Dashboard. The following results reflect observed console output, not claims of exhaustive coverage.

Test	Observed result
CMake configuration and C++17 build	Passed with Apple Clang.
Login and dashboard	Valid configured credential reached the Central Dashboard; invalid credential was rejected.
Input validation	Invalid numeric input and unknown menu choice displayed error messages.
Staff workflow	Fictional staff record could be added and displayed.
Patient care workflow	Appointment scheduling, priority display, check-up, admission, nurse display, and note update worked.
Emergency workflow	Registration, restart, triage display, discharge, invoice output, and emergency CSV persistence worked.
Navigation	Entrance to ICU returned a 60 m route.
Search	Prefix car returned Cardiology.
Presentation package	Existing four-slide PPTX passed package integrity checks.

Build and Run

From the project root: cmake -S . -B build, then cmake --build build --parallel 2, then ./build/HospitalApp on macOS or Linux. Windows users can run build\HospitalApp.exe after building.

9. Limitations and Future Scope

The current project uses a plain-text local password for demonstration only. CSV storage is not a transaction-safe database and has no encryption, role-based authorization, or concurrent-user protection. Viewing history is in-memory for the current session. Trie search starts with fixed terms and does not automatically refresh from staff changes. Future work can add secure authentication, a database, dynamic search indexing, a graphical interface, and automated unit and integration tests.

10. Conclusion

The Hospital Management System demonstrates how object-oriented classes and core data structures can support selected hospital workflows in C++17. The application provides a practical classroom demonstration of AVL trees, heaps, hash maps, stacks, queues, tries, graphs, and linked lists while keeping the implementation small enough to explain during a viva.

References

1. Project source code and CMake configuration in the accompanying repository.
2. C++17 standard library documentation for containers, memory, filesystem, and algorithms.
3. CMake documentation for configuring and building C++ projects.
4. Original repository attribution: Kianis534/Hospital-Management-System-cpp.

Personal placeholders to complete before submission: [Student Name], [Roll Number], [Faculty Guide Name], [Designation], [Department Name], [College Name], [20XX-20XX], and [Date].